

Datenstrukturen, Algorithmen und Programmierung 1

Vorlesung 24

Johannes Fischer

TU Dortmund

Map auf Basis von BST

```
1  template <typename U, typename V> struct Map : public BST<KVPair<U, V>> {
2      // operator[] for put:
3      V& operator[](U const& key) {
4          BSTNode<KVPair<U, V>>* where = BST<KVPair<U, V>>::find(KVPair<U, V>(key));
5          if (!where) {
6              where = BST<KVPair<U, V>>::insert(KVPair<U, V>(key));
7          }
8          return where->cont_.value_;
9      }
10
11     // read-only access:
12     V at(U const& key) const {
13         BSTNode<KVPair<U, V>>* where = BST<KVPair<U, V>>::find(KVPair<U, V>(key));
14         if (where)
15             return where->cont_.value_;
16         else
17             return {}; // Grund für 'return by value' (nicht Referenz)
18     }
19
20     bool find(U const& key) const {
21         BSTNode<KVPair<U, V>>* where = BST<KVPair<U, V>>::find(KVPair<U, V>(key));
22         if (where)
23             return true;
24         else
25             return false;
26     }
27     void insert(KVPair<T, U>) = delete; // so können geerbte Funktionen gelöscht werden, falls gewünscht
28 };
```

Anwendung: Worthäufigkeiten

Schreibe ein Programm, das für einen Text T eine Statistik über alle vorkommenden Wörter in T erstellt, d.h. für alle Wörter $w \in T$ zählt, wie oft w in T vorkommt.

- Beispiel:
 - Eingabe: $T =$ To be or not to be, that is the question.
 - Ausgabe: To : 1, be : 2, or : 1, not : 1, to : 1, that : 1, ...

Lösung mit Map

```
1  dap1::Map<dap1::MyString, int> freq; // map words to their frequencies
2
3  // walk through file and parse/count words:
4  FILE* in_file = fopen("faust.txt", "r");
5  dap1::MyString word;
6  for (unsigned char ch = fgetc(in_file); !feof(in_file); ch = fgetc(in_file)) {
7      if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
8          word.append(ch); // word has more chars -- requires (simple) new member function void append(char) in MyString
9      else { // finished reading word
10         if (word.get_size() > 0) freq[word]++;
11         word = "";
12     }
13 }
14 fclose(in_file);
15
16 // print statistics:
17 int n = freq.get_size();
18 printf("File contains %i different words\n", n);
19
20 // print all using vector:
21 dap1::ResizableArray<dap1::KeyValuePair<dap1::MyString, int>> sorted_alph = freq.get_sorted(); // inherited from dap1::BST
22 for (int i = 0; i < sorted_alph.get_size(); ++i)
23     printf("%s: %i\n", sorted_alph[i].key_.c_str(), sorted_alph[i].value_);
```

- Zusatzaufgabe: die häufigsten 10 Wörter ausgeben
 - Schwierigkeit: Paare nach **Wert** sortieren

- NB: `std::map` hat leicht anderes Interface

Heaps und Priority Queues

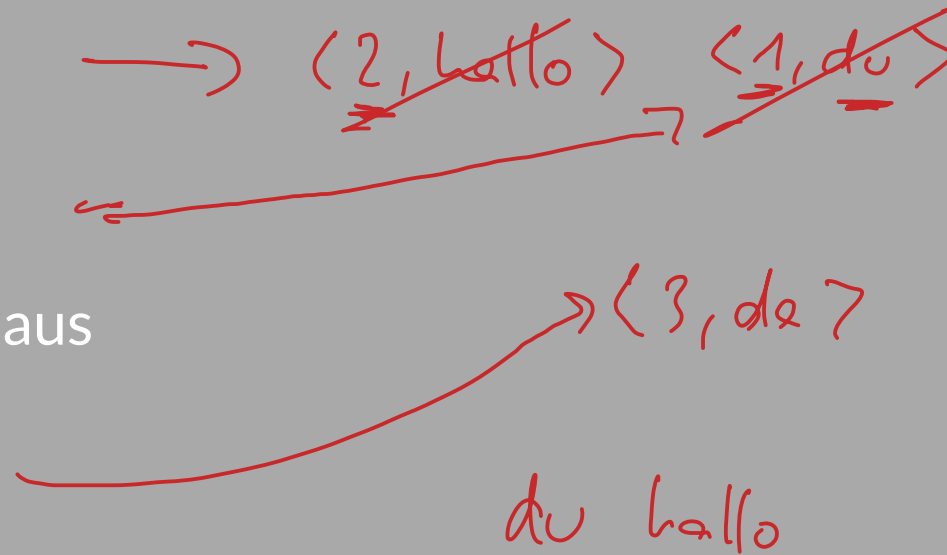
ADT Priority Queue (“Prioritätswarteschlange”)

- Eine **Prioritätswarteschlange** (engl. **priority queue**, “PQ”) mit Schlüsseln vom Typ K und Werten vom Typ V hat folgende Operationen:
 - $init(pq_name, capacity, K, V)$: initialisiert eine leere PQ mit Namen pq_name für bis zu $capacity$ viele key-value-Paare vom Typ (K, V)
 - $push(pq_name, \langle k, v \rangle)$: fügt den Wert v (vom Typ V) mit Schlüssel k (vom Typ K) in pq_name ein, falls die Kapazität noch nicht erschöpft ist
 - $extract_min(pq_name)$: löscht ein key-value-Paar $\langle k, v \rangle$ mit **minimalem** Schlüsselwert k aus der PQ und gibt es zurück ($k = \min\{q : \langle q, w \rangle \in pq_name\}$)
 - $is_empty(pq_name)$: gibt wahr zurück gdw. pq_name leer ist
 - $size(pq_name)$: gibt die Anzahl der key-value-Paare in pq_name zurück
- Wie eine **Queue**, aber pop liefert die Elemente aufsteigend geordnet nach “Priorität”
 - analog: **Maximum-PQs** (absteigend geordnet)

Quiz PriorityQueue

In welcher Reihenfolge werden die Elemente durch folgende Operationen ausgegeben:

- `init(pq, 10, int, std::string)`
- `push(pq, <2, hallo>)`
- `push(pq, <1, du>)`
- gib den Wert von `extract_min(pq)` aus
- `push(pq, <3, da>)`
- gib den Wert von `extract_min(pq)` aus



A: hallo du da

B: du hallo

C: hallo du

D: du da

E: nicht eindeutig

Quiz PriorityQueue

In welcher Reihenfolge werden die Elemente durch folgende Operationen ausgegeben:

- `init(pq, 10, int, std::string)`
- `push(pq, <2, hallo>)`
- `push(pq, <1, du>)`
- gib den Wert von `extract_min(pq)` aus
- `push(pq, <2, da>)`
- gib den Wert von `extract_min(pq)` aus

A: hallo du da

B: du hallo

C: hallo du

D: du da

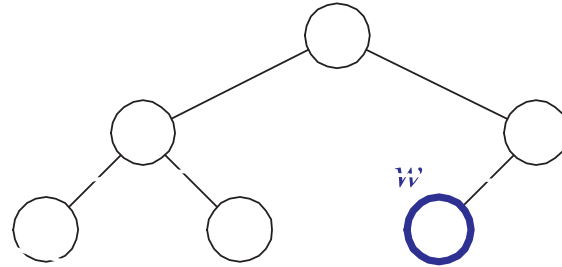
E: nicht eindeutig

Heaps

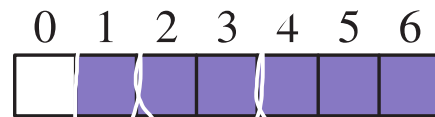
- ein **Min-Heap** (dt. Haufen/Halde) ist ein binärer Baum B für n key-value-Paare mit den 2 folgenden Eigenschaften:
 1. **Heap-Ordnung:** der Schlüssel aller Knoten v (außer der Wurzel) ist **nicht kleiner** als der Schlüssel ihres Elters
 2. **Vollständigkeit:** der Baum B ist **vollständig**, d.h. alle Tiefen (bis auf die letzte) sind voll besetzt, und die letzte Tiefe ist “von links nach rechts” besetzt
- Konsequenz: minimaler Schlüssel an Wurzel

Heaps haben eine Höhe von $\approx \log n$

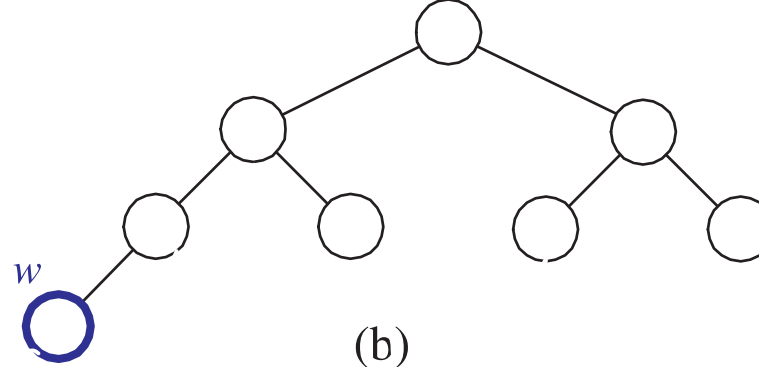
Array-Darstellung von Heaps



(a)



(c)



(b)



(d)

© Goodrich et al.: Data Structures and Algorithms in C++ (2nd ed.). John Wiley and Sons, 2011.

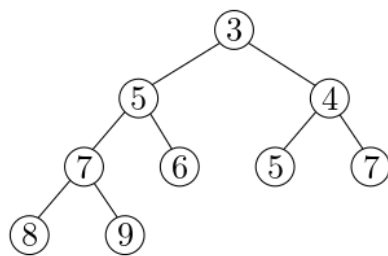
- “implizite Datenstruktur”

- Elter von Knoten k an Position $\lfloor k/2 \rfloor$
- Kinder von k an Positionen $2k$ und $2k + 1$ (falls vorhanden, d.h. $\leq n$)

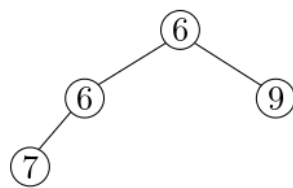
Inhalt von PQ<int> pq mit &pq=2 ?

```
1 template <typename T> class PQ {  
2     T* array_; int n_;  
3  
4     public: PQ(int max_size) : array_ = new T[max_size + 1], n_{0} {}  
5 };
```

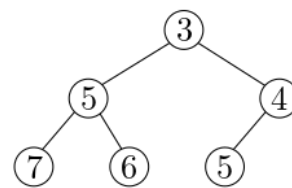
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
Speicher		2	6	6	9	7	1	3	5	4	7	6	5	7	8	9	



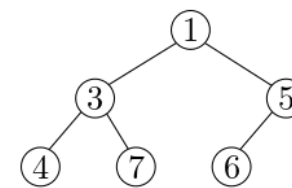
(a)



(b)



(c)



(d)

A: (a)

B: (b)

C: (c)

D: (d)

E: nicht eindeutig

Heap-Implementierung

```
namespace dap1 {
template <typename K, typename V>
class PQ {
    struct KVPair { // private "innere Klasse"
        K key_;
        V value_;

        KVPair(K const& key = {}, V const& val = {}) : key_{key}, value_{val} {}

        // implement operators to use keys:
        bool operator<(KVPair const& other) const { return key_ < other.key_; }
        bool operator==(KVPair const& other) const { return key_ == other.key_; }
    };

    KVPair* array_;
    int n_;

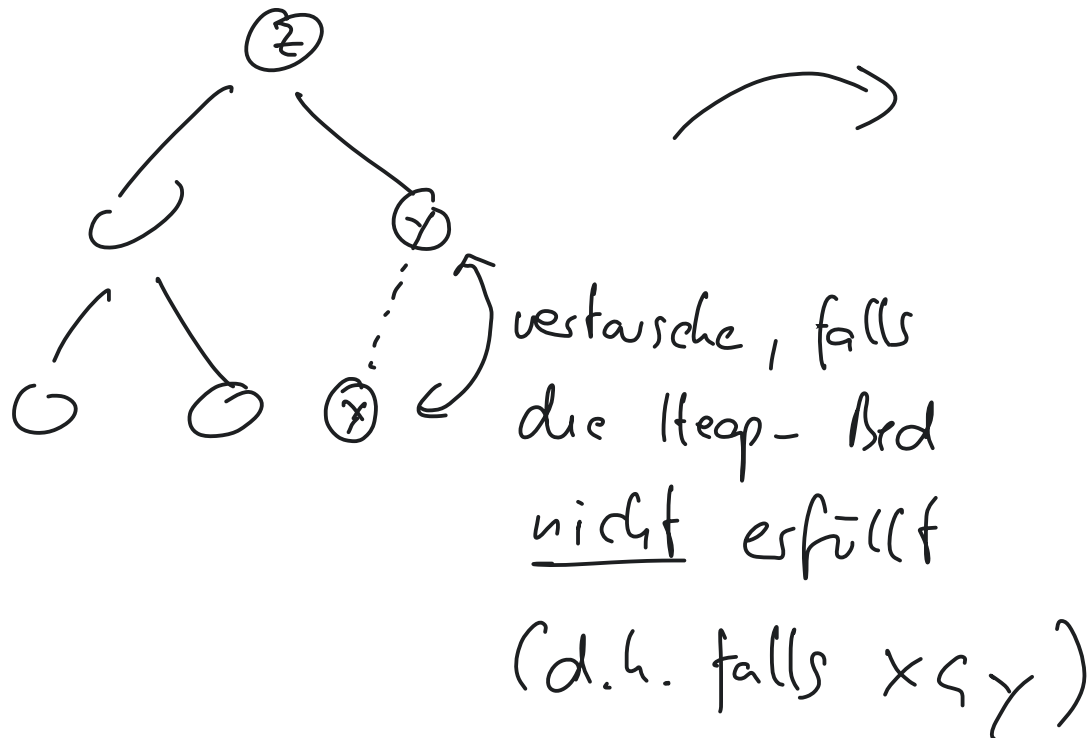
    // ...
};
```

- NB: KVPair ist eine **innere Klasse** von PQ (ansonsten wie bei Map)
 - d.h. kann nur innerhalb von PQ benutzt werden
 - ist generisch (Typen U, V), aber innerhalb von PQ ohne Template-Parameter benutzbar (KVPair und nicht KVPair<U, V>)

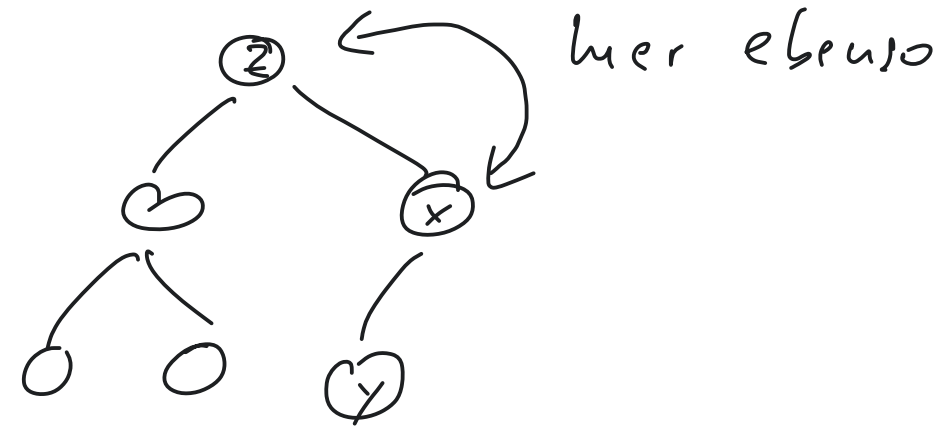
Heap-Operationen

- durch die Höhe $\lceil \log n \rceil$ implementieren Heaps den ADT PQ schnell:

$\text{push}(\langle \overset{\text{key}}{x}, \overset{\text{value}}{p} \rangle)$



$\text{extract_min}()$



PQ-Operationen mit Heaps in C++

```
1 public:
2   PQ(int capacity) : array_{new KVPair[capacity + 1]}, n_{0} {} // Konstruktor
```

```
1 public:
2   void push(K const& k, V const& v) {
3     // erzeuge "neuen Knoten" als neues letztes Blatt:
4     array_[++n_] = KVPair(k, v);
5     // jetzt Heap-Bedingung wiederherstellen:
6     sift_up(n_);
7   }
8 private:
9   void sift_up(int k) {
10    while (k > 1 && array_[k] < array_[k / 2]) {
11      swap(k, k / 2);
12      k = k / 2;
13    }
14  }
15
16  // swap array_[i] with array_[j]
17  void swap(int i, int j) {
18    T tmp = array_[i];
19    array_[i] = array_[j];
20    array_[j] = tmp;
21  }
```

```
1 public:
2   V extract_min() {
3     // nächste VL
4   }
```

